Artificial Intelligence (AI-2002)

Assignment 2: Constraint Satisfaction, Adversarial Search & Probability

Analytical Report

Name	Amnn Aijaz
Roll No.	24K-2584
Section	BDS-3A
Submission date	10-Nov-2026

Contents

Part A - Map colouring of Pakistan as a CSP (50 marks)
Part B - Isolation: alpha-beta search and heuristic design (50 marks) Part C - Probability: inference by enumeration (30 marks)
Appendix - Repository layout, how to run, verification log

Part A - Constraint Satisfaction Problem

A.1 Formulation

Variables X = {AK, BA, GB, ICT, KP, PB, SD}. **Domains** D(x) = {Red, Green, Blue} for every x. **Constraints** one binary constraint x ≠ y for every pair of bordering regions (adjacency lists exactly as given in the assignment; the code asserts they are symmetric). The constraint graph has 7 nodes and 11 edges; the triangles PB-KP-AK, BA-PB-KP, KP-PB-ICT, KP-AK-GB and BA-PB-SD each force three different colours, so 3 colours is the minimum.

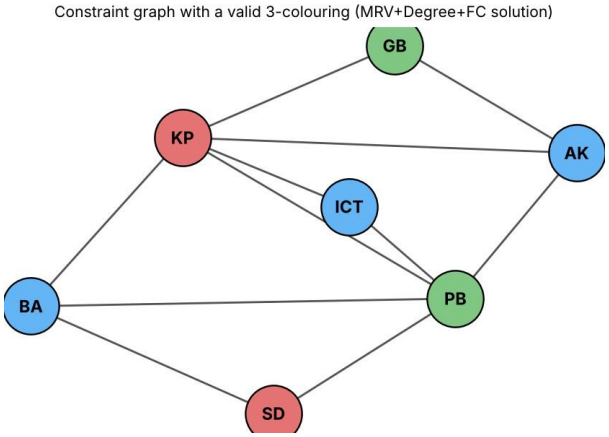


Figure 1 - Constraint graph and one valid colouring.

A.2 Task 1 - Basic backtracking search

Standard depth-first backtracking. Variables are taken in fixed alphabetical order (AK, BA, GB, ICT, KP, PB, SD); colours are tried in the order Red, Green, Blue. A colour is tested against the already assigned neighbours only, so a doomed partial assignment is noticed late (no look-ahead). Counters: assignments, backtracks, recursive calls and constraint checks.

Pseudo-code

```
BT(assignment):
  if |assignment| = 7: return success
  var <- first unassigned var
  iable (alphabetical) for va
  lue in [Red, Green, Blue]:
     if value consistent with as
        signed neighbours:assig
        nment[var] <- value ; as
        signments++ if BT(assig
        nment) succeeds: return s
        uccessremove assignment
        [var] ; backtracks++
  return failure
```

A.3 Task 2 - MRV + Degree heuristic + Forward Checking

MRV picks the unassigned variable with the fewest legal colours left. **Degree** breaks MRV ties by choosing the variable with the most constraints on other unassigned variables (it constrains the rest of the problem most). **Forward checking** removes the chosen colour from the domains of all unassigned neighbours immediately after each assignment; if any domain becomes empty the branch is abandoned at once and the pruned values are restored when we backtrack. Order of decisions on the first run is shown below.

Solution found: KP=Red, PB=Green, AK=Blue, BA=Blue, GB=Green, ICT=Blue, SD=Red.

A.4 Task 3 - Performance analysis

(*i*) First solution, alphabetical order, colours R-G-B.

Metric	Basic backtracking	MRV + Degree + FC
Assignments	7	7
Backtracks	0	0
Recursive calls	8	8
Constraint checks	33	11
Values pruned by FC	0	11

On this particular ordering both solvers are lucky and find a solution with zero backtracks (alphabetical order happens to be a friendly ordering and R-G-B happens to work), so the only visible gain is 3x fewer constraint checks. A single run therefore says little; the following experiments are more informative.

(*ii*) Exhaustive search (all solutions). Forcing the search to explore the whole tree exposes its real size. Both solvers find the same 6 solutions (= 3! colourings of the PB-KP-AK triangle; every other region is then forced), but the effort differs a lot. (Backtracks here also count the undo after each complete solution.)

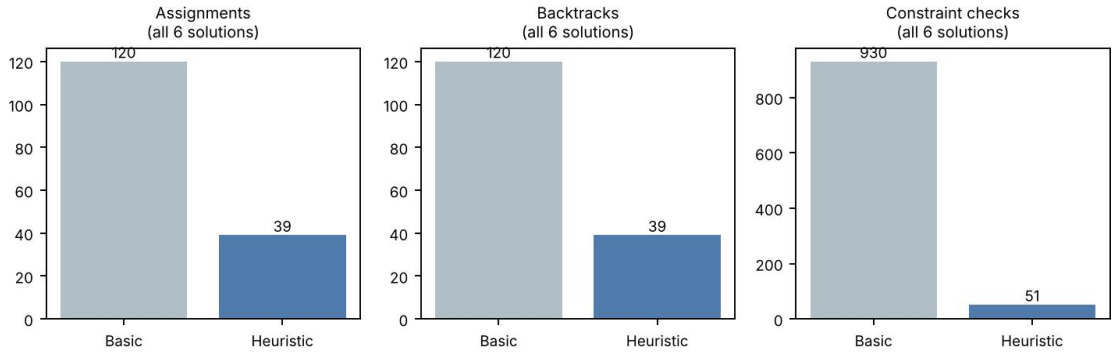


Figure 2 - Search effort for enumerating all solutions.

Metric	Basic	Heuristic	Reduction
Assignments	120	39	3.1x
Backtracks	120	39	3.1x
Recursive calls	121	40	3.0x
Constraint checks	930	51	18.2x

(*iii*) Robustness over all 30,240 (variable order x colour order) combinations (7! orders x 3! colour orders, first solution each). Basic backtracking depends heavily on the ordering it is given; the heuristic solver chooses its own variable order, so only the colour order matters.

Average per run	Basic	Heuristic
Assignments	13.23	7.00
Backtracks	6.23	0.00
Recursive calls	14.23	8.00
Worst-case backtracks	50	0

***(iv)* Ablation (all solutions).** Which ingredient helps?

Configuration	Assignments	Backtracks	Calls
none (basic order, no FC)	120	120	121
FC only	102	102	67
MRV only	120	120	121
MRV + Degree	39	39	40
MRV + Degree + FC (full)	39	39	40

Discussion. (1) Without forward checking the domains never shrink, so MRV sees every variable with 3 colours and degenerates to list order (row 'MRV only' = baseline) - MRV needs forward checking to be informative. (2) With static domains the Degree tie-break is what actually orders the variables (MRV + Degree row); it already starts at the hub KP and works outwards through the triangles, cutting the tree from 120 to 39 assignments. (3) Forward checking detects dead ends one level early, so a doomed branch is cut as soon as a neighbour has an empty domain instead of after a futile attempt at that variable; it also reduces constraint-check cost (930 -> 51). (4) Over all orderings the full heuristic solver never backtracks (0.00 vs 6.23 on average and up to 50 for the baseline): it always assigns 7 variables in 8 calls, which is the minimum possible. The map is tiny, so wall-clock time is negligible (microseconds) and slightly higher for the heuristic solver because of its bookkeeping; the benefit would grow quickly with problem size.

Part B - Adversarial Search: Isolation

B.1 Game model

5x5 board; AI = player 0 at (0,0) and moves first, Human = player 1 at (4,4). Moves are **chess-queen** moves (any distance along a row, column or diagonal) that may not cross or land on a destroyed cell or on the opponent's token. The cell a player has just left is destroyed permanently. A player with no legal move on his/her turn loses. State = (position of both tokens, bitmask of destroyed cells, player to move); the bitmask makes 'available spaces' a single integer and successor generation cheap.

Why queen moves? Queen mobility fragments the 5x5 board quickly, giving games of about 20 plies and a branching factor of 11 at the first ply, so a depth-4 search is a real limitation and evaluation quality matters - ideal for comparing heuristics.

B.2 Alpha-beta engine (depth limit 4 plies)

The AI runs depth-limited minimax with alpha-beta pruning. At a MAX node (AI to move) alpha is raised to the best value found so far; at a MIN node beta is lowered; **the loop over children stops as soon as alpha ≥ beta** (the remaining siblings cannot change the decision). Terminal positions (side to move has no move) return ±(1000 + remaining depth) so that quicker wins and slower losses are preferred; at depth 0 the heuristic scores the leaf. The same function drives both players: `me` is the player we choose a move for, and evaluations are always from `me`'s point of view.

```
ALPHABETA(s, depth, alpha, beta):
 if s has no legal move:          return -/+
 (1000+depth) # side to move loses if dept
 h == 0:          return h(s)
 if MAX t
    o mov
    e: v = -
    inf
    for m in moves(s): v = max(v, ALPHABETA(a
                pply(s,m), depth-1, alpha, beta))
                alpha = max(alpha, v)
                if alpha >= beta: bre
 ak                                  # cut
 offelse: symmetric with min / beta
```

Verification of pruning. At the start position, for every depth 1-4, alpha-beta returns exactly the same value as plain minimax (asserted in code) while expanding far fewer nodes:

Depth	Minimax nodes	Alpha-beta nodes	Cut-offs (alpha>=beta)	Saving

1	12	12	0	0.0%
2	125	59	8	52.8%
3	1,319	365	38	72.3%
4	12,871	1,213	226	90.6%

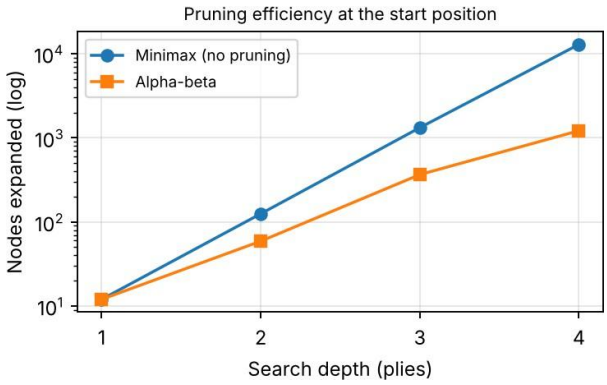


Figure 3 - Nodes expanded vs depth (log scale).

B.3 Heuristic evaluation functions

Function 1 - linear mobility differential. $h_1(s)$ = (#legal moves of me) - (#legal moves of opponent). Cheap (two move generations) and captures the immediate freedom of each token, but it is short-sighted: a cell that can be reached by me today may be unreachable after one more destruction, and it ignores who 'owns' the remaining space.

Function 2 - Voronoi territory (custom metric). For every free cell compute, by breadth-first search over queen moves, the number of moves each player needs to reach it. The cell is counted for the player who gets there first (ties go to the player whose turn it is). $h_2(s)$ = #cells I own - #cells the opponent owns. It is a 'territorial block weight': cells I can reach alone are worth a full point and contested cells go to the faster player. This measures how much of the board each side can claim, which is what decides Isolation end-games once the tokens separate.

Experimental set-up. Both agents search to depth 4 with identical alpha-beta code, so the only difference is the evaluation function. Because play is otherwise deterministic, each game starts with 2 random plies (one move per side) to create variety; 50 games with H2 first and 50 with H2 second (100 games, fixed seed 2026). Each heuristic also plays 100 games against a uniformly random mover as a sanity check.

	H1 mobility	H2 territory
Wins (of 100 head-to-head games)	36	64
Win rate	36%	64%
Wins vs random mover (of 100)	97	99
Avg. nodes expanded per game	2668	2696
Avg. think time per game (s)	0.013	0.127



Figure 4 - Heuristic comparison.

B.4 Comparison and conclusions

Strength. The territory heuristic beat mobility in 64 of 100 games (64%), including from both the first and the second player seat. Both easily beat a random mover (97/100 and 99/100), so the gap between them is the meaningful result. **Why H2 wins:** with only 4 plies of lookahead the AI cannot see the end of the game; mobility counts adjacent freedom but cannot tell whether those moves lead into a dead-end pocket, whereas territory approximates the end-game (who will have more squares left to walk on) and therefore steers the AI to partition the board in its favour.

Cost. Both searches expand about the same number of nodes (~2696 per game) because depth and pruning are identical, but H2 needs two BFS sweeps per leaf, making it about 10x slower in wall-clock time (0.13s vs 0.01s per game) - still negligible on a 5x5 board. **Limitations / improvements.** H2 ignores that the opponent can still block cells during the race; a natural improvement is to combine it with mobility (e.g. h = territory + w * mobility), or to add move-ordering (try the highest-scoring successor first) to prune even more. The result is statistical (100 games, random openings) and depends on the queen-move rule and depth limit 4.

Verification log. A complete deterministic example game (H2 moves first against H1) is stored in `results/part_b_showcase_game.txt`; the winner of the two no-random showcase games is player 0 when H2 moves first and player 1 (= H2) when H2 moves second; i.e. H2 wins both deterministic games.

Part C - Probability: Inference by Enumeration

The network (Re→Bu, At→Ov, Bu→Cr, Ov→Cr, Cr→Al, At→Al, Cr→Co, Al→Es, Co→Es) gives the joint distribution

P(Re,At,Bu,Ov,Cr,Al,Co,Es) = P(Re) P(At) P(Bu|Re) P(Ov|At) P(Cr|Bu,Ov) P(Al|Cr,At) P(Co|Cr) P(Es|Al,Co).

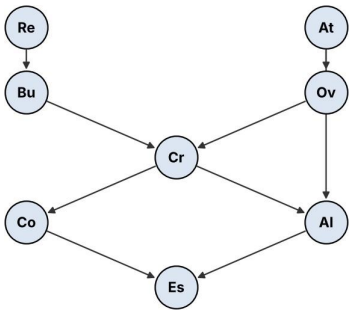


Figure 5 - The Bayesian network.

Inference by enumeration sums this product over all hidden variables and normalises. Hidden variables that are not ancestors of the query/evidence sum to 1 and drop out (for example Es and Co when they are not in the query/evidence), which is how the sums below are shortened. Every answer was also computed by the program (full recursive ENUMERATION-ASK) and cross-checked by brute-force summation of all $2^8 = 256$ joint entries - the results agree to 12 decimals.

Preliminary marginals

P(Bu) = P(Re)P(Bu|Re) + P(¬Re)P(Bu|¬Re) = 0.2(0.3) + 0.8(0.05) = 0.1000
P(Ov) = P(At)P(Ov|At) + P(¬At)P(Ov|¬At) = 0.1(0.6) + 0.9(0.1) = 0.1500
(Bu and Ov are independent: their ancestors Re and At are independent root variables.)

(a) P(Cr)

$P(Cr) = \sum_{Bu,Ov} P(Bu)\, P(Ov)\, P(Cr|Bu,Ov)$

Bu	Ov	P(Bu)	P(Ov)	P(Cr|Bu,Ov)	Product
T	T	0.1000	0.1500	0.9500	0.014250
T	F	0.1000	0.8500	0.6000	0.051000
F	T	0.9000	0.1500	0.4000	0.054000
F	F	0.9000	0.8500	0.0300	0.022950
				Sum	0.142200

P(Cr) = 0.1422

(b) P(Al)

First the joint $P(Cr,At) = P(At) \sum_{Ov} P(Ov|At)\, P(Cr|Ov)$, with $P(Cr|Ov) = \sum_{Bu} P(Bu)P(Cr|Bu,Ov)$: P(Cr|Ov) = 0.1(0.95)+0.9(0.4) = 0.4550; P(Cr|¬Ov) = 0.1(0.6)+0.9(0.03) = 0.0870.

P(Cr,At) = 0.1[0.6(0.455)+0.4(0.087)] = 0.03078; P(Cr,¬At) =

$0.9[0.1(0.455)+0.9(0.087)] = 0.11142$. Check: sum = 0.1422 = P(Cr). Then $P(\neg Cr,At) = 0.1 - 0.03078 = 0.06922$; $P(\neg Cr,\neg At) = 0.9 - 0.11142 = 0.78858$.

$P(Al) = \sum_{Cr,At} P(Cr,At)\, P(Al|Cr,At)$

Cr	At	P(Cr,At)	P(Al|Cr,At)	Product
T	T	0.03078	0.9900	0.030472
T	F	0.11142	0.9000	0.100278
F	T	0.06922	0.7000	0.048454
F	F	0.78858	0.0500	0.039429
			Sum	0.218633

P(Al) = 0.2186

(c) P(Cr | Re)

Given Re, At is unaffected so P(Ov) = 0.15 still holds, and P(Bu|Re) = 0.3. $P(Cr|Re) = \sum_{Bu,Ov} P(Bu|Re)\, P(Ov)\, P(Cr|Bu,Ov)$

Bu	Ov	P(Bu|Re)	P(Ov)	P(Cr|Bu,Ov)	Product
T	T	0.3000	0.1500	0.9500	0.042750
T	F	0.3000	0.8500	0.6000	0.153000
F	T	0.7000	0.1500	0.4000	0.042000
F	F	0.7000	0.8500	0.0300	0.017850
				Sum	0.255600

P(Cr | Re) = 0.2556 (compared with P(Cr) = 0.1422: a new release raises crash risk).

(d) P(Es)

$P(Es) = \sum_{Al,Co} P(Al,Co)\, P(Es|Al,Co)$, with $P(Al,Co) = \sum_{Cr,At} P(Cr,At)\, P(Al|Cr,At)\, P(Co|Cr)$ (using the P(Cr,At) values from part (b)).

Al	Co	P(Al,Co)	P(Es|Al,Co)	Product
T	T	0.119926	0.9500	0.113930
T	F	0.098707	0.6000	0.059224
F	T	0.086724	0.4000	0.034690
F	F	0.694643	0.0100	0.006946
		1.000000	Sum	0.214790

(The four P(Al,Co) entries add to 1.0000, and their Al=True entries add to P(Al) = 0.2186 from part (b).)

P(Es) = 0.2148

(e) P(At | Al, ¬Co)

$P(At \mid Al,\neg Co) = \alpha \sum_{Cr} P(Cr,At)\, P(Al|Cr,At)\, P(\neg Co|Cr)$ (Es is summed out = 1).

At	Cr	P(Cr,At)	P(Al|Cr,At)	P(¬Co|Cr)	Product
T	T	0.03078	0.9900	0.1500	0.004571
T	F	0.06922	0.7000	0.9000	0.043609
F	T	0.11142	0.9000	0.1500	0.015042

F	F	0.78858	0.0500	0.9000	0.035486

Unnormalised: At = 0.048179, ¬At = 0.050528; α = 1/(0.048179 + 0.050528) = 10.1310.

P(At | Al, ¬Co) = 0.048179 / 0.098707 = 0.4881 (prior P(At) was only 0.1: an alert without complaints strongly suggests an attack, since an attack can fire the alert even when no crash happens).

(f) P(Re | Cr, ¬Al)

P(Re,Cr,¬Al) = $\sum_{At,Bu,Ov}$ P(Re) P(At) P(Bu|Re) P(Ov|At) P(Cr|Bu,Ov) P(¬Al|Cr,At) (Co and Es sum to 1 and vanish). Terms for Re and ¬Re:

At	Bu	Ov	Term, Re = T	Term, Re = F
T	T	T	0.00003420	0.00002280
T	T	F	0.00001440	0.00000960
T	F	T	0.00003360	0.00018240
T	F	F	0.00000168	0.00000912
F	T	T	0.00051300	0.00034200
F	T	F	0.00291600	0.00194400
F	F	T	0.00050400	0.00273600
F	F	F	0.00034020	0.00184680
		Sum	0.00435708	0.00709272

P(Re | Cr, ¬Al) = 0.004357 / (0.004357 + 0.007093) = 0.3805 (the prior P(Re) = 0.2 rises: a crash is mostly explained by a bug, and a release makes a bug more likely).

Summary of answers

Query	Answer (4 d.p.)
(a) P(Cr)	0.1422
(b) P(Al)	0.2186
(c) P(Cr | Re)	0.2556
(d) P(Es)	0.2148
(e) P(At | Al, ~Co)	0.4881
(f) P(Re | Cr, ~Al)	0.3805

Appendix - Repository layout and verification

```
AI-2002-Assignment-2/
|-- README.md
|-- part_a_csp/csp_solver.py      # Part A(Tasks 1-3)
|-- part_b_isolation/isolation.py # Part B   (alpha-beta, H1, H2, pl
ay-vs-human)
|-- part_c_probability/bayes_enumeration.py  # Part C (inference by
enumeration)
|-- report/AI2002_Assignment2_Report.pdf     # this report
|-- report/figures/*.png
`-- results/                  # saved console output of each program
```

How to run

```
python part_a_csp/csp_solver.py
python part_b_isolation/isolation.py --games 50    # experiments
python part_b_isolation/isolation.py --pla
y                                  # play against
the AIpython part_c_probability/bayes_enumer
ation.py
```

Verification checks built into the code

- Part A: adjacency symmetry asserted; every returned colouring is re-checked by an independent verifier; both solvers return the same number of solutions (6).
- Part B: alpha-beta value is asserted equal to plain minimax value at depths 1-4 (pruning loses nothing); terminal losses are detected when the side to move has no legal move.
- Part C: every answer is asserted equal between recursive enumeration and brute-force summation of all 256 joint entries; the hand-worked tables above are asserted equal to the program's output.